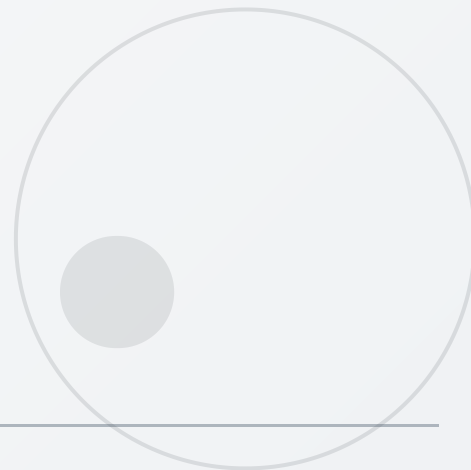




Especificaciones Funcionales y Técnicas

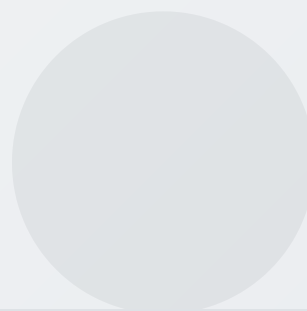
Sistema de Comandas por Voz con Reconocimiento Offline VOSK
para Restaurantes y Centros de Consumo

Documento de Arquitectura: VOSK-Comandas-v1.0

Fecha: Junio 2026

Clasificación: Documento Técnico de Implementación

Plataforma: PHP / MariaDB / Ubuntu 22.04 LTS



Documento elaborado con base en investigación técnica del motor VOSK, arquitectura frugal de bajo costo operativo y especificaciones validadas para entornos de producción en microempresas gastronómicas.

Indice de Contenidos

1. Resumen Ejecutivo

2. Alcance y Objetivos del Sistema

2.1. Proposito

2.2. Alcance Funcional

2.3. Usuarios del Sistema

3. Especificaciones Funcionales

3.1. Modulo de Captura por Voz (VOSK)

3.2. Modulo de Registro de Comandas

3.3. Modulo de Pantalla de Cocina (KDS)

3.4. Modulo de Pantalla de Caja

3.5. Modulo de Administracion

4. Especificaciones Tecnicas

4.1. Arquitectura del Sistema

4.2. Stack Tecnologico

4.3. Modelo de Base de Datos

4.4. API Backend (PHP)

4.5. Frontend del Mesero (PWA)

5. Flujos de Trabajo

5.1. Flujo de Toma de Comanda

5.2. Flujo de Preparacion en Cocina

5.3. Flujo de Cobro en Caja

6. Interfaces de Usuario

6.1. Interfaz del Mesero (Android/PWA)

6.2. Interfaz de Pantalla de Cocina

6.3. Interfaz de Pantalla de Caja

7. Requerimientos de Infraestructura

7.1. Servidor Local (Ubuntu 22.04)

7.2. Dispositivos Android

7.3. Red Local

8. Seguridad y Resiliencia

9. Plan de Implementacion

10. Glosario de Terminos

11. Referencias

1. Resumen Ejecutivo

El presente documento establece las **especificaciones funcionales y tecnicas completas** para el desarrollo e implementacion de un sistema de comandas por reconocimiento de voz offline, basado en el motor **VOSK (Voice Open Source Kit)**, orientado a restaurantes, bares y centros de consumo que operan con infraestructura tecnologica frugal de bajo costo.

El sistema permite que los meseros, mediante sus telefonos moviles Android, dicten las ordenes de los clientes de forma natural en lenguaje espanol. El motor VOSK, ejecutandose localmente dentro del navegador del dispositivo mediante **WebAssembly (WASM)**, transcribe el audio a texto sin requerir conexion a internet ni servicios de nube. Las comandas transcritas se envian via HTTP a un **servidor local** con **PHP 8.x y MariaDB** sobre **Ubuntu 22.04 LTS**, donde se registran con numero de mesa, hora de captura e identificacion del mesero. Posteriormente, las ordenes se visualizan en tiempo real en pantallas ubicadas en la cocina (sistema KDS) y en la caja para cobro.

La arquitectura propuesta elimina completamente los costos operativos por consumo de APIs de reconocimiento de voz en la nube, garantiza la **privacidad total de los datos** (el audio nunca abandona el dispositivo), y opera de forma autonoma ante fallas de conexion a internet. El modelo de lenguaje en espanol `vosk-model-small-es-0.42` ocupa aproximadamente **39 MB** comprimido y se almacena en cache persistente via IndexedDB tras la primera descarga, habilitando el funcionamiento 100% offline en visitas subsecuentes.

Principales Ventajas del Sistema

- **Costo operativo cero** en reconocimiento de voz: sin APIs de Google, Azure ni OpenAI.
- **Funcionamiento offline completo**: el procesamiento de audio ocurre integralmente en el dispositivo del mesero.
- **Infraestructura minima**: un solo servidor local con PHP/MariaDB y red Wi-Fi basica.
- **Escalabilidad por numero de meseros**: cada dispositivo distribuye la carga de procesamiento.
- **Resiliencia ante fallas de red**: cola offline en IndexedDB con sincronizacion automatica.

2. Alcance y Objetivos del Sistema

2.1. Proposito

El sistema tiene como proposito principal **automatizar el proceso de captura y registro de comandas** en restaurantes mediante tecnologia de reconocimiento de voz offline, eliminando la dependencia de anotaciones manuales en papel o la navegacion por interfaces tactiles complejas durante las horas pico de servicio. Al mismo tiempo, busca establecer un flujo digital continuo entre el area de comedor (meseros), el area de preparacion (cocina) y el area de cobro (caja), mediante pantallas dedicadas que presentan la informacion de las ordenes en tiempo real con los datos minimos necesarios para identificar el orden de atencion.

2.2. Alcance Funcional

El sistema abarca las siguientes funcionalidades principales:

Tabla 1. Alcance Funcional del Sistema de Comandas VOSK

Modulo		Funcionalidad		Descripcion
Captura Voz	por	Dictado comandas	de	El mesero dicta la orden en lenguaje natural; VOSK transcribe el audio a texto localmente en el navegador del telefono Android.
Captura Voz	por	Gramatica restringida		El vocabulario del reconocedor se limita al catalogo de productos del restaurante para maximizar la precision.
Registro Comandas	de	Asociacion mesa	de	Cada comanda se registra con el numero de mesa correspondiente.
Registro Comandas	de	Identificacion mesero	de	El sistema registra el nombre o identificador del mesero que capturo la orden.
Registro Comandas	de	Marca temporal		Cada orden almacena la fecha y hora exacta de captura (timestamp).
Pantalla Cocina (KDS)		Visualizacion ordenes	de	Lista de comandas pendientes con datos minimos: numero de mesa, productos, hora, estado.
Pantalla Cocina (KDS)		Gestion de estados		Los cocineros marcan ordenes como "en preparacion" y "listo".

Pantalla Caja	Resumen de consumo	de	Muestra el total acumulado por mesa para agilizar el cobro.
Pantalla Caja	Cierre de cuenta		Permite cerrar la cuenta de una mesa e imprimir el ticket.
Administracion	Catalogo de productos	de	Alta, baja y modificacion de productos, precios y categorias.
Administracion	Reportes basicos		Ventas por periodo, productos mas vendidos, rendimiento por mesero.

2.3. Usuarios del Sistema

Tabla 2. Perfiles de Usuarios del Sistema

Rol	Dispositivo	Funciones Principales
Mesero	Telefono Android (propio o corporativo)	Dictar comandas por voz; visualizar confirmacion de orden registrada; ver historial de sus comandas del turno.
Cocinero / Chef	Pantalla dedicada (monitor/tablet) en cocina	Visualizar comandas pendientes ordenadas por hora; marcar ordenes como en preparacion o listas.
Cajero	Pantalla de caja (PC o tablet)	Visualizar consumo por mesa; procesar cobros; cerrar cuentas.
Administrador / Dueno	PC de escritorio o laptop	Configurar catalogo de productos; gestionar usuarios; consultar reportes de ventas y estadisticas.

3. Especificaciones Funcionales

3.1. Modulo de Captura por Voz (VOSK)

El modulo de captura por voz constituye el nucleo diferenciador del sistema. Se ejecuta integramente dentro del navegador web del telefono Android del mesero mediante tecnologia WebAssembly, sin requerir instalacion de aplicaciones nativas ni conexion a servicios en la nube.

3.1.1. Inicializacion del Motor VOSK

Al acceder por primera vez a la pagina PHP desde su telefono Android, el mesero debe completar los siguientes pasos de inicializacion:

1. **Descarga del modelo de lenguaje:** El navegador descarga automaticamente el archivo comprimido `vosk-model-small-es-0.42.tar.gz` (aproximadamente 39 MB) desde el servidor local. Esta descarga ocurre una unica vez y el modelo se almacena en la **IndexedDB** del navegador para accesos futuros inmediatos.
2. **Carga en memoria:** El modelo se descomprime y carga en la memoria RAM del dispositivo, consumiendo entre **150 MB y 250 MB** adicionales durante el reconocimiento activo.
3. **Configuracion de gramatica restringida:** El sistema limita el vocabulario de reconocimiento a los terminos del catalogo del restaurante (nombres de productos, numeros, comandos de control), elevando la precision de captura por encima del 95% en entornos con ruido moderado.

3.1.2. Proceso de Dictado

El mesero inicia la captura de una comanda presionando un boton en la interfaz PWA. El sistema activa el microfono del dispositivo y procesa el audio en tiempo real siguiendo esta secuencia:

1. El mesero dicta la orden en lenguaje natural: *"Mesa cinco, dos tacos al pastor con todo y una Coca-Cola"*.
2. El **AudioWorklet** del navegador captura el audio a **16 kHz en formato PCM mono** y lo transmite al Web Worker donde VOSK procesa las muestras.
3. VOSK devuelve el texto reconocido en tiempo real, mostrando resultados parciales mientras el mesero habla y un resultado final al detectar pausa.
4. El sistema aplica **correccion fonetica fuzzy** (algoritmo Levenshtein) para mapear palabras similares a productos del catalogo (ejemplo: "pastol" → "pastor").
5. El mesero visualiza el texto transcrito en pantalla y puede confirmar, repetir o corregir antes del envio definitivo.

3.1.3. Especificaciones del Modelo VOSK

Tabla 3. Especificaciones Tecnicas del Modelo VOSK para Espanol

Parametro	Valor	Observacion
-----------	-------	-------------

Modelo	vosk-model-small-es-0.42	Version optimizada para dispositivos moviles
Tamano comprimido	~39 MB	Descarga unica, almacenada en IndexedDB
Tamano descomprimido	~50 MB	Consumo en disco del navegador
Consumo RAM	150 - 250 MB	Durante reconocimiento activo
Idioma soportado	Espanol (Latinoamerica y Espana)	Incluye variaciones dialectales comunes
Frecuencia de muestreo	16 kHz PCM mono	Requerimiento del pipeline de audio
Latencia de respuesta	< 500 ms	Desde fin del habla hasta texto final
Licencia	Apache 2.0	Uso comercial permitido sin restricciones

3.1.4. Optimizaciones de Rendimiento

Para garantizar la estabilidad del sistema en dispositivos Android de gama media-baja, se implementan las siguientes optimizaciones tecnicas:

- **Carga perezosa (Lazy Loading):** El modelo WASM no se descarga automaticamente al abrir la pagina. La descarga y activacion del Web Worker se inician unicamente cuando el mesero presiona el boton de captura por primera vez.
- **Terminacion controlada del Web Worker:** Al finalizar el turno, el mesero puede liberar explicitamente la memoria ocupada por el modelo mediante un boton de apagado que ejecuta `worker.terminate()` y establece la referencia del modelo a `null`, devolviendo el consumo de RAM a 0 MB.
- **AudioContext como singleton:** Se utiliza una unica instancia global del contexto de audio inicializada a 16 kHz, que se suspende y reanuda entre usos en lugar de destruirse y recrearse, evitando fugas de memoria.
- **Prevencion de apagado de pantalla:** Mediante la API `navigator.wakeLock`, el sistema solicita un bloqueo de suspension mientras el motor de dictado esta activo, evitando que Android detenga el procesamiento de WebAssembly.
- **Wake lock de audio:** Reproduccion de una pista de audio muda en bucle durante el reconocimiento para notificar al kernel que la pestana opera multimedia activa, previniendo que el recolector de procesos en segundo plano termine el Web Worker.

3.2. Modulo de Registro de Comandas

Una vez transcrita la comanda a texto, el sistema procesa la informacion y la persiste en la base de datos central. Este modulo gestiona el ciclo de vida completo de una orden desde su captura hasta su cierre.

3.2.1. Datos Minimos de Registro

Cada comanda registrada en el sistema debe contener obligatoriamente los siguientes campos:

Tabla 4. Campos Obligatorios del Registro de Comandas

Campo	Tipo	Descripcion	Ejemplo
numero_mesa	INT	Numero identificador de la mesa atendida	5, 12, 3
hora_captura	DATETIME	Fecha y hora exacta de la captura de la comanda	2026-06-11 14:32:18
id_mesero	INT / VARCHAR	Identificador del mesero que realizo la captura	JUAN_P, MARIA_G
texto_transcrito	TEXT	Texto completo tal como fue transcrito por VOSK	"mesa cinco dos tacos al pastor con todo y una coca"
productos	JSON / ARRAY	Lista de productos identificados del catalogo con cantidades	[{"id": 3, "cant": 2}, {"id": 15, "cant": 1}]
estado	ENUM	Estado actual de la comanda en el flujo	pendiente, en_preparacion, listo, entregado, cancelado

3.2.2. Estados de una Comanda

El ciclo de vida de una comanda atraviesa los siguientes estados, controlados por transiciones definidas:

Tabla 5. Maquina de Estados de la Comanda

Estado	Descripcion	Quien Actua	Transicion Siguiente
Pendiente	Comanda registrada, a la espera de ser atendida por cocina	Sistema automatico	En preparacion

En preparacion	Un cocinero ha tomado la orden y esta trabajando en ella	Cocinero (pantalla KDS)	Listo
Listo	Los productos de la comanda estan preparados	Cocinero (pantalla KDS)	Entregado
Entregado	Los productos fueron entregados a la mesa	Mesero (confirma entrega)	Cobrado (al cierre)
Cobrado	La cuenta de la mesa ha sido pagada	Cajero	Estado final
Cancelado	La comanda fue anulada por error o solicitud del cliente	Mesero / Administrador	Estado final

3.2.3. Cola Offline y Sincronizacion

En caso de perdida de conexion Wi-Fi momentanea, el sistema implementa una **cola offline** mediante IndexedDB que garantiza que ninguna comanda se pierda:

1. Al detectar ausencia de conectividad (`navigator.onLine === false`), la comanda se almacena localmente en la tabla `cola_pedidos` de IndexedDB con estado `"pendiente_sync"`.
2. Se establece un listener del evento `"online"` que dispara automaticamente el proceso de sincronizacion cuando la red se restaura.
3. La sincronizacion procesa las comandas en orden FIFO (primero en entrar, primero en salir), enviando cada una via POST al endpoint PHP.
4. Las comandas sincronizadas exitosamente se eliminan de la cola local. Si una sincronizacion falla, permanece en cola para reintento.

3.3. Modulo de Pantalla de Cocina (KDS)

El Kitchen Display System (KDS) es la interfaz digital ubicada en el area de cocina que reemplaza los tradicionales tickets impresos en papel. Su proposito es presentar las comandas pendientes de forma clara, ordenada y en tiempo real, permitiendo a los cocineros gestionar eficientemente el flujo de preparacion.

3.3.1. Requisitos de Visualizacion Minimos

La pantalla de cocina debe mostrar un **banner minimo** con la informacion esencial para identificar el orden de atencion. Cada tarjeta de comanda debe incluir:

- **Numero de mesa:** En formato grande y prominente (fuente mayor al resto del texto) para identificacion rapida a distancia.
- **Hora de captura:** Marca temporal que permite determinar la antigüedad de la orden y priorizar comandas mas viejas.
- **Nombre del mesero:** Identificacion de quien tomo la orden para consultas o aclaraciones.
- **Lista de productos:** Descripcion concisa de los articulos solicitados con sus cantidades.
- **Tiempo transcurrido:** Contador dinamico que muestra cuantos minutos y segundos han pasado desde la captura.
- **Indicador de prioridad por color:**Codigo visual que cambia segun el tiempo de espera (verde < 10 min, amarillo 10-20 min, rojo > 20 min).

3.3.2. Orden de Atencion y Priorizacion

Las comandas se presentan ordenadas estrictamente por **hora de captura** (FIFO), de modo que la orden mas antigua aparece primero en la lista. No se permite reordenamiento manual; la prioridad es siempre cronologica para garantizar equidad en los tiempos de espera de los clientes.

La interfaz debe refrescarse automaticamente cada **5 segundos** mediante polling al endpoint PHP, o mediante tecnologia Server-Sent Events (SSE) si el servidor lo soporta. Al recibir una nueva comanda, el sistema puede emitir una alerta sonora breve para notificar a los cocineros.

3.3.3. Interaccion del Cocinero

La pantalla de cocina es una interfaz de solo lectura con capacidad de actualizacion de estado limitada:

- **Boton "Preparar":** El cocinero selecciona una comanda y la marca como "En preparacion". Esto notifica al sistema que la orden esta siendo atendida.
- **Boton "Listo":** Al terminar la preparacion, el cocinero marca la comanda como "Lista". El sistema notifica al mesero correspondiente (via vibracion o notificacion en su PWA) que la orden esta lista para recoger.
- **Boton "Recuperar":** Permite recuperar comandas marcadas como listas por error, devolviendolas al estado "En preparacion".

3.4. Modulo de Pantalla de Caja

La pantalla de caja proporciona al cajero una vision consolidada del consumo por mesa para agilizar el proceso de cobro y cierre de cuentas.

3.4.1. Visualizacion Minima de Caja

El banner de la pantalla de caja debe mostrar:

- **Listado de mesas activas:** Solo mesas con comandas pendientes o consumos no cobrados.
- **Numero de mesa y total acumulado:** Suma de todos los productos ordenados por esa mesa.
- **Hora de la primera comanda:** Indica cuanto tiempo lleva la mesa ocupada.
- **Mesero asignado:** Nombre del mesero que atiende la mesa.
- **Boton "Ver Detalle":** Expande la informacion para mostrar el desglose itemizado de productos y cantidades.
- **Boton "Cobrar":** Permite registrar el pago y cerrar la cuenta de la mesa.

3.4.2. Proceso de Cobro

1. El cajero selecciona la mesa a cobrar de la lista de mesas activas.
2. El sistema muestra el desglose completo: productos, cantidades, precios unitarios y total general.
3. El cajero ingresa el metodo de pago (efectivo, tarjeta, transferencia) y, si aplica, el monto recibido.
4. El sistema calcula el cambio si es necesario, registra la transaccion y cambia el estado de todas las comandas de esa mesa a "Cobrado".
5. Se genera un ticket de cierre que puede imprimirse o enviarse por WhatsApp al cliente si asi lo solicita.

3.5. Modulo de Administracion

Este modulo proporciona las herramientas necesarias para configurar y mantener el sistema operativo.

3.5.1. Gestion del Catalogo de Productos

- **Alta de productos:** Nombre, descripcion, precio unitario, categoria, codigo interno.
- **Modificacion:** Edicion de precios, nombres y disponibilidad.
- **Baja logica:** Desactivacion de productos sin eliminarlos del historial.
- **Categorias:** Organizacion por tipo (entradas, platos fuertes, bebidas, postres, etc.).

3.5.2. Gestion de Usuarios

- Creacion de perfiles de meseros con nombre de usuario y contrasena.
- Asignacion de roles: mesero, cocinero, cajero, administrador.
- Control de activacion/desactivacion de usuarios.

3.5.3. Reportes Basicos

Tabla 6. Reportes Disponibles en el Modulo de Administracion

Reporte	Contenido	Periodicidad
Ventas por dia	Total de ventas, numero de transacciones, ticket promedio	Diario, por rango de fechas
Productos vendidos	mas Ranking de productos por cantidad vendida e ingresos	Diario, semanal, mensual
Rendimiento mesero	por Numero de comandas, ventas generadas, tiempo promedio de atencion	Semanal, quincenal, mensual
Tiempos de preparacion	de Tiempo promedio desde captura hasta "listo" por categoria de producto	Semanal, mensual
Ventas por hora	Distribucion de ventas a lo largo del dia para planificacion de personal	Diario, promedio historico

4. Especificaciones Tecnicas

4.1. Arquitectura del Sistema

El sistema opera bajo una **arquitectura de tres capas** con procesamiento distribuido: la capa de presentacion (PWA en Android), la capa de logica de negocio (PHP en servidor local) y la capa de datos (MariaDB). El reconocimiento de voz se ejecuta en el edge (dispositivo del

mesero) mediante WebAssembly, distribuyendo la carga computacional y eliminando la dependencia de servicios cloud.

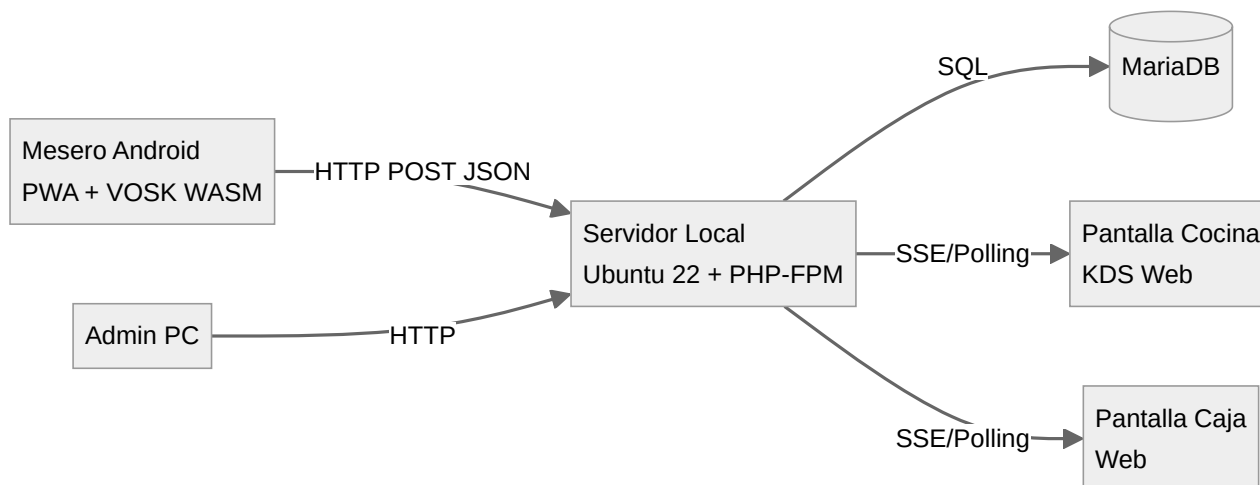
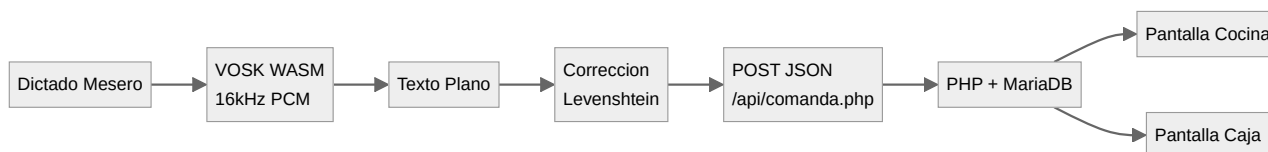


Figura 1. Diagrama de arquitectura de alto nivel del sistema de comandos VOSK

4.1.1. Diagrama de Flujo de Datos

El flujo de datos desde el dictado del mesero hasta la visualización en cocina y caja sigue el siguiente recorrido:

1. **Captura de audio:** El mesero dicta la comanda; el AudioWorklet del navegador captura audio PCM a 16 kHz mono.
2. **Procesamiento local:** El Web Worker de VOSK (WASM) transcribe el audio a texto plano usando el modelo de lenguaje almacenado en IndexedDB.
3. **Corrección fonética:** El algoritmo Levenshtein mapea palabras del dictado contra el catálogo de productos, corrigiendo errores menores de transcripción.
4. **POST al servidor:** La PWA envía un JSON via HTTPS POST al endpoint `/api/comanda.php` con: texto transcrito, número de mesa, id del mesero y productos parseados.
5. **Validación y persistencia:** PHP valida los datos, ejecuta el procedimiento almacenado `RegistrarPedidoVoz` que inserta la comanda y detalles dentro de una transacción atómica.
6. **Notificación en tiempo real:** Las pantallas de cocina y caja, conectadas via SSE (Server-Sent Events) o polling cada 5 segundos, reciben la nueva comanda y la renderizan.
7. **Actualización de estado:** Los cocineros interactúan con la pantalla KDS para cambiar estados; los cambios se reflejan en tiempo real en todas las pantallas conectadas.

**Figura 2.** Flujo de datos end-to-end del sistema

4.2. Stack Tecnologico

Tabla 7. Stack Tecnologico Completo del Sistema

Capa	Tecnologia	Version	Proposito
Sistema operativo servidor	Ubuntu Server LTS	22.04	Infraestructura base del servidor local
Servidor web	Apache HTTP Server	2.4+	Host de aplicacion PHP, SSL, virtual hosts
Lenguaje backend	PHP	8.1 - 8.3	Logica de negocio, API REST, procedimientos almacenados
Procesador PHP	PHP-FPM	8.1 - 8.3	Gestion eficiente de procesos PHP concurrentes
Base de datos	MariaDB	10.6+	Persistencia relacional de comandas, productos y usuarios
Frontend mesero	PWA (HTML5 + JS)	ES2020+	Interfaz progresiva, captura de voz, trabajo offline
Reconocimiento de voz	VOSK (WebAssembly)	0.3.45+	Transcripcion offline de audio a texto
Modelo de lenguaje	vosk-model-small-es-0.42	0.42	Modelo pre-entrenado en espanol para dispositivos moviles
Audio API	Web Audio API + AudioWorklet	W3C standard	Captura y procesamiento de audio de baja latencia
Almacenamiento local	IndexedDB API	W3C standard	Cache del modelo VOSK y cola offline de comandas
Sincronizacion	Fetch API + Background Sync	W3C standard	Envio de comandas y sincronizacion offline/online
Notificaciones tiempo real	Server-Sent Events (SSE)	W3C standard	Push de nuevas comandas a pantallas cocina y caja

4.2.1. Configuracion de PHP-FPM

Para optimizar el rendimiento en el servidor local y manejar multiples conexiones simultaneas de meseros, se recomienda la siguiente configuracion en `/etc/php/8.3/fpm/pool.d/www.conf` (ajustar segun la version instalada):

```
pm = ondemand
pm.max_children = 10
pm.process_idle_timeout = 10s
pm.max_requests = 500
```

Esta configuracion utiliza el modo `ondemand` que crea procesos hijo solo cuando hay peticiones activas, ideal para un restaurante con picos de actividad variables. El valor de `max_children = 10` permite atender hasta 10 peticiones concurrentes, suficiente para la mayoría de los establecimientos con 3-8 meseros operando simultaneamente.

4.2.2. Configuracion de MariaDB

En el archivo `/etc/mysql/mariadb.conf.d/50-server.cnf`, se recomiendan los siguientes ajustes para un servidor dedicado a la aplicacion de comandas:

```
innodb_buffer_pool_size = 512M
innodb_log_file_size = 128M
innodb_flush_log_at_trx_commit = 2
query_cache_size = 0
query_cache_type = 0
```

El `innodb_buffer_pool_size` de 512 MB asigna suficiente memoria RAM para cachear tablas frecuentes (comandas activas, catalogo de productos). El valor `innodb_flush_log_at_trx_commit = 2` mejora el rendimiento de escritura al no forzar flush en cada commit, aceptando un riesgo minimo de perdida de datos ante corte electrico (mitigado con UPS recomendado).

4.3. Modelo de Base de Datos

El esquema relacional se disena para soportar el registro eficiente de comandas, el seguimiento de estados y la generacion de reportes operativos. A continuacion se detallan

las tablas principales:

4.3.1. Tabla: usuarios

```
CREATE TABLE usuarios (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  usuario VARCHAR(50) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  rol ENUM('mesero', 'cocinero', 'cajero', 'administrador') DEFAULT 'mesero',  
  activo TINYINT(1) DEFAULT 1,  
  creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
) ENGINE=InnoDB;
```

4.3.2. Tabla: mesas

```
CREATE TABLE mesas (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  numero INT UNIQUE NOT NULL,  
  capacidad INT DEFAULT 4,  
  activa TINYINT(1) DEFAULT 1,  
  creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
) ENGINE=InnoDB;
```

4.3.3. Tabla: categorias

```
CREATE TABLE categorias (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(50) NOT NULL,  
  orden INT DEFAULT 0,  
  activa TINYINT(1) DEFAULT 1  
) ENGINE=InnoDB;
```

4.3.4. Tabla: productos

```
CREATE TABLE productos (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  categoria_id INT,  
  nombre VARCHAR(100) NOT NULL,  
  descripcion TEXT,  
  precio DECIMAL(10,2) NOT NULL,  
  codigo VARCHAR(20),  
  disponible TINYINT(1) DEFAULT 1,  
  palabras_clave VARCHAR(255),  
  creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  actualizado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  FOREIGN KEY (categoria_id) REFERENCES categorias(id)  
) ENGINE=InnoDB;
```

El campo `palabras_clave` almacena sinonimos y variaciones foneticas separadas por comas para enriquecer el mapeo del reconocedor de voz (ejemplo: "coca, coca-cola, coca cola, refresco").

4.3.5. Tabla: comandas (pedidos)

```
CREATE TABLE comandas (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  mesa_id INT NOT NULL,  
  mesero_id INT NOT NULL,  
  texto_transcrito TEXT,  
  total DECIMAL(10,2) DEFAULT 0.00,  
  estado ENUM('pendiente', 'en_preparacion', 'listo', 'entregado', 'cobrado',  
  'cancelado') DEFAULT 'pendiente',  
  creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  actualizado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  FOREIGN KEY (mesa_id) REFERENCES mesas(id),  
  FOREIGN KEY (mesero_id) REFERENCES usuarios(id)  
) ENGINE=InnoDB;
```

4.3.6. Tabla: detalle_comandas

```
CREATE TABLE detalle_comandas (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  comanda_id INT NOT NULL,  
  producto_id INT NOT NULL,  
  cantidad INT NOT NULL DEFAULT 1,  
  precio_unitario DECIMAL(10,2) NOT NULL,  
  subtotal DECIMAL(10,2) NOT NULL,  
  notas VARCHAR(255),  
  FOREIGN KEY (comanda_id) REFERENCES comandas(id) ON DELETE CASCADE,  
  FOREIGN KEY (producto_id) REFERENCES productos(id)  
) ENGINE=InnoDB;
```

4.3.7. Tabla: pagos

```
CREATE TABLE pagos (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  mesa_id INT NOT NULL,  
  total DECIMAL(10,2) NOT NULL,  
  metodo_pago ENUM('efectivo', 'tarjeta', 'transferencia') NOT NULL,  
  monto_recibido DECIMAL(10,2),  
  cambio DECIMAL(10,2),  
  cajero_id INT NOT NULL,  
  pagado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (mesa_id) REFERENCES mesas(id),  
  FOREIGN KEY (cajero_id) REFERENCES usuarios(id)  
) ENGINE=InnoDB;
```

4.3.8. Procedimiento Almacenado: RegistrarComanda

```

DELIMITER //
CREATE PROCEDURE RegistrarComanda(
    IN p_mesa_id INT,
    IN p_mesero_id INT,
    IN p_texto_transcrito TEXT,
    IN p_json_productos JSON
)
BEGIN
    DECLARE v_comanda_id INT;
    DECLARE v_total DECIMAL(10,2) DEFAULT 0.00;
    DECLARE v_producto_id INT;
    DECLARE v_cantidad INT;
    DECLARE v_precio DECIMAL(10,2);
    DECLARE i INT DEFAULT 0;
    DECLARE n INT;

    START TRANSACTION;

    -- Crear cabecera de comanda
    INSERT INTO comandas (mesa_id, mesero_id, texto_transcrito, total)
    VALUES (p_mesa_id, p_mesero_id, p_texto_transcrito, 0.00);
    SET v_comanda_id = LAST_INSERT_ID();

    -- Procesar productos del JSON
    SET n = JSON_LENGTH(p_json_productos);
    WHILE i < n DO
        SET v_producto_id = JSON_UNQUOTE(JSON_EXTRACT(p_json_productos, CONCAT('$[',
i, '].producto_id')));
        SET v_cantidad = JSON_UNQUOTE(JSON_EXTRACT(p_json_productos, CONCAT('$[', i,
'].cantidad')));

        SELECT precio INTO v_precio FROM productos WHERE id = v_producto_id AND
disponible = 1;

        IF v_precio IS NOT NULL THEN
            INSERT INTO detalle_comandas (comanda_id, producto_id, cantidad,
precio_unitario, subtotal)
            VALUES (v_comanda_id, v_producto_id, v_cantidad, v_precio, v_precio *
v_cantidad);
            SET v_total = v_total + (v_precio * v_cantidad);
        END IF;

        SET i = i + 1;
    END WHILE;

    -- Actualizar total de la comanda
    UPDATE comandas SET total = v_total WHERE id = v_comanda_id;

```

```

DELIMITER //
CREATE PROCEDURE RegistrarComanda(
    IN p_mesa_id INT,
    IN p_mesero_id INT,
    IN p_texto_transcrito TEXT,
    IN p_json_productos JSON
)
BEGIN
    DECLARE v_comanda_id INT;
    DECLARE v_total DECIMAL(10,2) DEFAULT 0.00;
    DECLARE v_producto_id INT;
    DECLARE v_cantidad INT;
    DECLARE v_precio DECIMAL(10,2);
    DECLARE i INT DEFAULT 0;
    DECLARE n INT;

    START TRANSACTION;

    -- Crear cabecera de comanda
    INSERT INTO comandas (mesa_id, mesero_id, texto_transcrito, total)
    VALUES (p_mesa_id, p_mesero_id, p_texto_transcrito, 0.00);
    SET v_comanda_id = LAST_INSERT_ID();

    -- Procesar productos del JSON
    SET n = JSON_LENGTH(p_json_productos);
    WHILE i < n DO
        SET v_producto_id = JSON_UNQUOTE(JSON_EXTRACT(p_json_productos, CONCAT('$[',
i, '].producto_id')));
        SET v_cantidad = JSON_UNQUOTE(JSON_EXTRACT(p_json_productos, CONCAT('$[', i,
'].cantidad')));

        SELECT precio INTO v_precio FROM productos WHERE id = v_producto_id AND
disponible = 1;

        IF v_precio IS NOT NULL THEN
            INSERT INTO detalle_comandas (comanda_id, producto_id, cantidad,
precio_unitario, subtotal)
            VALUES (v_comanda_id, v_producto_id, v_cantidad, v_precio, v_precio *
v_cantidad);
            SET v_total = v_total + (v_precio * v_cantidad);
        END IF;

        SET i = i + 1;
    END WHILE;

    -- Actualizar total de la comanda
    UPDATE comandas SET total = v_total WHERE id = v_comanda_id;

    COMMIT;

```

4.4. API Backend (PHP)

La API REST en PHP expone endpoints especificos para cada operacion del sistema. Todos los endpoints devuelven JSON y requieren autenticacion mediante token JWT o sesion PHP segun la criticidad.

4.4.1. Endpoint: Registro de Comanda

```
POST /api/comanda.php
Content-Type: application/json

{
  "mesa_id": 5,
  "mesero_id": 3,
  "texto_transcrito": "mesa cinco dos tacos al pastor con todo y una coca",
  "productos": [
    {"producto_id": 12, "cantidad": 2},
    {"producto_id": 45, "cantidad": 1}
  ]
}
```

```
RESPUESTA 200 OK:
{
  "status": "success",
  "comanda_id": 1284,
  "total": 145.00,
  "hora_registro": "2026-06-11 14:32:18",
  "mensaje": "Comanda registrada correctamente"
}
```

```
RESPUESTA 400 Bad Request:
{
  "status": "error",
  "mensaje": "Parametros incompletos: se requiere mesa_id y productos"
}
```

4.4.2. Endpoint: Consulta de Comandas Pendientes (Pantalla Cocina)

```
GET /api/comandas/pendientes.php?limite=50
```

RESPUESTA 200 OK:

```
{
  "status": "success",
  "total": 12,
  "comandas": [
    {
      "id": 1284,
      "numero_mesa": 5,
      "mesero": "Juan Perez",
      "hora_captura": "2026-06-11 14:32:18",
      "minutos_transcurridos": 4,
      "estado": "pendiente",
      "productos": ["2x Tacos al pastor", "1x Coca-Cola 600ml"]
    }
  ]
}
```

4.4.3. Endpoint: Actualizacion de Estado

```
PUT /api/comandas/estado.php
```

Content-Type: application/json

```
{
  "comanda_id": 1284,
  "estado": "en_preparacion"
}
```

RESPUESTA 200 OK:

```
{
  "status": "success",
  "comanda_id": 1284,
  "estado_anterior": "pendiente",
  "estado_nuevo": "en_preparacion",
  "actualizado_en": "2026-06-11 14:36:42"
}
```

4.4.4. Endpoint: Resumen por Mesa (Pantalla Caja)

GET /api/mesas/activas.php

RESPUESTA 200 OK:

```
{
  "status": "success",
  "mesas_activas": 8,
  "mesas": [
    {
      "mesa_id": 5,
      "numero_mesa": 5,
      "mesero": "Juan Perez",
      "primera_comanda": "2026-06-11 14:15:00",
      "minutos_ocupada": 22,
      "total_consumo": 485.00,
      "comandas_pendientes": 2,
      "comandas_listas": 1
    }
  ]
}
```

4.5. Frontend del Mesero (PWA)

La aplicacion para el mesero se implementa como una **Progressive Web App (PWA)** accesible mediante el navegador del telefono Android. No requiere instalacion desde una tienda de aplicaciones; el mesero accede a una URL en el servidor local y puede agregar el icono a su pantalla de inicio.

4.5.1. Estructura de la PWA

Tabla 8. Archivos Principales de la PWA del Mesero

Archivo	Funcion
index.html	Shell principal de la aplicacion con metadatos PWA
manifest.json	Configuracion de icono, nombre y tema de la PWA
sw.js	Service Worker para cache offline y sincronizacion en segundo plano
js/vosk.js	Wrapper de inicializacion del modelo VOSK
js/recognizer.js	Logica de AudioWorklet, captura PCM y comunicacion con VOSK

js/app.js	Logica principal: UI, envio de comandas, cola offline
js/fuzzy.js	Algoritmo de correccion fonetica Levenshtein
css/app.css	Estilos responsivos optimizados para pantallas moviles

4.5.2. Configuración de HTTPS Autofirmado

Dado que la API `getUserMedia` requiere un contexto seguro (HTTPS o localhost), el servidor Apache debe configurarse con un certificado SSL autofirmado para operación en la red local:

```
# Generar certificado autofirmado
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout /etc/ssl/private/comandas-local.key \
  -out /etc/ssl/certs/comandas-local.crt \
  -subj "/C=MX/ST=Estado/L=Ciudad/O=Restaurante/CN=comandas.local"

# Configurar virtual host en Apache
<VirtualHost *:443>
  ServerName comandas.local
  DocumentRoot /var/www/comandas

  SSLEngine on
  SSLCertificateFile /etc/ssl/certs/comandas-local.crt
  SSLCertificateKeyFile /etc/ssl/private/comandas-local.key

  <Directory /var/www/comandas>
    AllowOverride All
    Require all granted
  </Directory>
</VirtualHost>
```

Nota sobre Certificados Autofirmados

Los navegadores mostraran una advertencia de seguridad al acceder al sitio por primera vez. El usuario debe seleccionar "Avanzado" y "Proceder a comandas.local (no seguro)". Esta advertencia solo aparece la primera vez y puede omitirse agregando el certificado como excepcion de confianza en los ajustes del navegador Android.

5. Flujos de Trabajo

5.1. Flujo de Toma de Comanda

El flujo de trabajo principal inicia cuando un cliente realiza un pedido y el mesero utiliza su telefono para registrar la comanda por voz:

1. **Acceso a la PWA:** El mesero abre Chrome en su Android, navega a `https://comandas.local` e inicia sesion con sus credenciales.
2. **Seleccion de mesa:** En la pantalla principal, selecciona el numero de mesa que atiende de una cuadrícula visual.
3. **Inicio de dictado:** Presiona y mantiene el boton circular grande con icono de microfono. El sistema emite un tono corto indicando que el microfono esta activo.
4. **Dictado de la orden:** El mesero habla de forma natural: *"Dos tacos al pastor con todo, una coca de medio y un agua de horchata"*. El texto aparece en tiempo real en la pantalla conforme VOSK lo transcribe.
5. **Soltar y revisar:** Al soltar el boton, el sistema muestra el texto completo transcrito junto con los productos identificados del catalogo. Si un producto no fue reconocido, aparece resaltado para correccion manual.
6. **Confirmacion:** El mesero verifica los productos y cantidades, y presiona "Enviar comanda". El sistema almacena la orden y muestra confirmacion visual y haptica (vibracion).
7. **Comanda siguiente:** La interfaz regresa automaticamente a la seleccion de mesa para una nueva captura.

Todo el proceso de dictado y confirmacion esta disenado para completarse en **menos de 15 segundos** por comanda, comparado con 30-60 segundos promedio de navegacion tactil tradicional.

5.2. Flujo de Preparacion en Cocina

1. **Llegada de comanda:** La nueva orden aparece automaticamente en la pantalla de cocina con un sonido de notificacion breve.
2. **Visualizacion:** La comanda se muestra en la parte superior de la lista con fondo verde (menos de 10 minutos de espera).

3. **Toma de orden:** Un cocinero presiona la comanda para expandirla y luego el boton "Preparar". El estado cambia a "En preparacion" y la tarjeta se mueve a la seccion correspondiente.
4. **Preparacion:** El cocinero prepara los alimentos consultando los productos y notas de la comanda.
5. **Marcar como listo:** Al completar, presiona "Listo". La comanda desaparece de la pantalla principal y el sistema notifica al mesero.
6. **Recogida:** El mesero acude a la cocina, identifica su comanda por el numero y entrega los alimentos en la mesa.

5.3. Flujo de Cobro en Caja

1. **Solicitud de cuenta:** El cliente solicita la cuenta; el mesero (o el cliente directamente) indica el numero de mesa al cajero.
2. **Consulta en pantalla:** El cajero ubica la mesa en la pantalla de caja y presiona "Ver detalle".
3. **Revision:** El sistema muestra el desglose itemizado con productos, cantidades, precios unitarios y total. El cajero confirma con el cliente.
4. **Cobro:** El cajero presiona "Cobrar", selecciona el metodo de pago, ingresa el monto recibido y el sistema calcula el cambio.
5. **Confirmacion:** Al confirmar, todas las comandas de esa mesa cambian a estado "Cobrado", la mesa queda libre y el sistema puede imprimir un ticket opcional.

6. Interfaces de Usuario

6.1. Interfaz del Mesero (Android/PWA)

La interfaz del mesero se disena con principios de **diseño móvil primero**, optimizada para pantallas de 5.5 a 6.7 pulgadas y operacion con una sola mano. Los elementos tactiles tienen un area minima de 48x48 dp para facilitar la interaccion durante el movimiento.

6.1.1. Pantalla de Seleccion de Mesa

Pantalla inicial despues del inicio de sesion. Muestra una cuadricula de tarjetas numeradas representando las mesas del restaurante. Cada tarjeta indica:

- **Numero de mesa** en fuente grande centrada.
- **Indicador de estado:** Verde (libre), Rojo (ocupada), Naranja (comandas listas para entregar).
- **Numero de comandas activas** en esquina superior derecha si la mesa esta ocupada.

6.1.2. Pantalla de Dictado

Al seleccionar una mesa, la interfaz cambia a la pantalla de captura:

- **Barra superior:** Numero de mesa seleccionada y nombre del mesero.
- **Area de transcripcion:** Espacio amplio donde aparece el texto transcrito en tiempo real con fuente grande (18sp) para legibilidad a distancia.
- **Boton de dictado:** Boton circular grande en la parte inferior, centrado. Color primario del restaurante. Se mantiene presionado durante el dictado.
- **Lista de productos identificados:** Debajo del area de transcripcion, tarjetas horizontales con cada producto reconocido, cantidad y precio.
- **Boton "Enviar":** Barra inferior fija con boton de ancho completo para confirmar y enviar la comanda.

6.1.3. Pantalla de Confirmacion

Tras enviar la comanda, se muestra una pantalla breve (2 segundos) con:

- Icono de check grande en color verde.
- Texto "Comanda enviada".
- Numero de comanda asignado.
- Tiempo estimado de preparacion.

6.2. Interfaz de Pantalla de Cocina

La pantalla de cocina es una interfaz web optimizada para monitores horizontales de 19 a 24 pulgadas, visualizada a distancia de 1 a 2 metros. El diseño prioriza la **legibilidad extrema** y la identificación rápida.

6.2.1. Layout Principal

- **Encabezado fijo:** Logo del restaurante, fecha/hora actual, numero total de comandas pendientes y comandas en preparacion.

- **Area principal:** Tarjetas de comanda dispuestas en cuadrícula (grid) de 2 o 3 columnas según el tamaño del monitor.
- **Barra lateral derecha** (opcional): Lista de comandas "Listas" esperando recogida.

6.2.2. Diseño de Tarjeta de Comanda

Cada tarjeta ocupa aproximadamente un tercio del ancho de la pantalla y contiene:

Tabla 9. Elementos Visuales de la Tarjeta de Comanda (KDS)

Elemento	Formato Visual	Propósito
Numero de mesa	Fuente extra-grande (48-72px), negrita, centrada	Identificación inmediata desde lejos
Tiempo transcurrido	Contador MM:SS, color cambiante	Priorizar ordenes mas antiguas
Nombre del mesero	Fuente mediana (18px), esquina superior	Contacto para aclaraciones
Hora de captura	Fuente pequena (14px), formato 24h	Referencia temporal exacta
Lista de productos	Bullets con cantidad y nombre, fuente grande	Saber que preparar
Indicador de color	Borde izquierdo grueso: verde/amarillo/rojo	Identificar urgencia a glance
Botones de accion	"Preparar" y "Listo", fuente grande, alto contraste	Interaccion rapida sin precision extrema

6.2.3. Código de Colores por Tiempo

Tabla 10. Código de Colores de Prioridad en Pantalla de Cocina

Color	Tiempo Transcurrido	Significado
Verde	0 - 10 minutos	Tiempo normal de espera
Amarillo / Naranja	10 - 20 minutos	Tiempo de espera prolongado, atender pronto
Rojo	Mas de 20 minutos	Urgente, cliente puede estar insatisfecho

6.3. Interfaz de Pantalla de Caja

La pantalla de caja presenta un listado limpio y escaneable de las mesas activas, optimizado para la operacion rapida del cajero.

6.3.1. Layout de Pantalla de Caja

- **Encabezado:** Titulo "Caja", fecha/hora, total de ventas del dia.
- **Lista de mesas activas:** Tarjetas horizontales apiladas verticalmente.
- **Area de detalle:** Al seleccionar una mesa, panel derecho con desglose completo.

6.3.2. Tarjeta de Mesa en Caja

Cada fila de mesa muestra:

- **Numero de mesa:** Fuente grande, negrita.
- **Total acumulado:** Fuente grande, alineada a la derecha, formato monetario.
- **Mesero:** Nombre del responsable.
- **Tiempo ocupada:** Desde la primera comanda.
- **Comandas:** Contador de activas vs. listas.
- **Boton "Cobrar":** Destacado en color primario.

7. Requerimientos de Infraestructura

7.1. Servidor Local (Ubuntu 22.04)

El servidor central opera sobre **Ubuntu Server 22.04 LTS** instalado en hardware de escritorio estandar o una mini PC. Este equipo funciona como host de la aplicacion PHP, base de datos MariaDB y servidor web Apache con SSL.

Tabla 11. Especificaciones Minimas y Recomendadas del Servidor

Componente	Minimo	Recomendado
Procesador (CPU)	Intel Core i3 / AMD Ryzen 3 (4 nucleos)	Intel Core i5 / AMD Ryzen 5 (6 nucleos)
Memoria RAM	4 GB	8 GB

Almacenamiento	120 GB SSD	256 GB SSD NVMe
Red	Ethernet Gigabit + Wi-Fi 802.11ac	Ethernet Gigabit + Wi-Fi 6 (802.11ax)
Puertos USB	4 x USB 3.0	6 x USB 3.0 (para expansion)
Sistema operativo	Ubuntu Server 22.04 LTS	Ubuntu Server 22.04 LTS (actualizado)

7.1.1. Software Base Requerido

```
# Instalacion de dependencias del sistema
sudo apt update && sudo apt upgrade -y
sudo apt install -y apache2 php8.3 php8.3-fpm php8.3-mysql php8.3-json \
    php8.3-mbstring php8.3-curl mariadb-server mariadb-client \
    openssl ssl-cert

# Habilitar modulos de Apache
sudo a2enmod ssl rewrite proxy proxy_fcgi setenvif
sudo a2enconf php8.3-fpm
sudo systemctl restart apache2
```

7.2. Dispositivos Android

Tabla 12. Requerimientos de Hardware para Telefonos Android de Meseros

Especificacion	Minimo	Recomendado
Sistema operativo	Android 8.0 (Oreo, API 26)	Android 11+ (API 30)
Memoria RAM	3 GB	4 GB o superior
Almacenamiento libre	200 MB	500 MB
Procesador	64-bit, Octa-core 1.8 GHz	Snapdragon 6-series o equivalente
Navegador	Chrome 90+	Chrome 120+ (actualizado)
Conectividad	Wi-Fi 802.11n	Wi-Fi 5 (802.11ac) o superior

7.3. Red Local

El sistema opera integramenente dentro de una **red local privada**. No requiere conexion a internet para su funcionamiento basico, aunque una conexion ocasional es util para actualizaciones del sistema.

7.3.1. Topologia de Red Recomendada

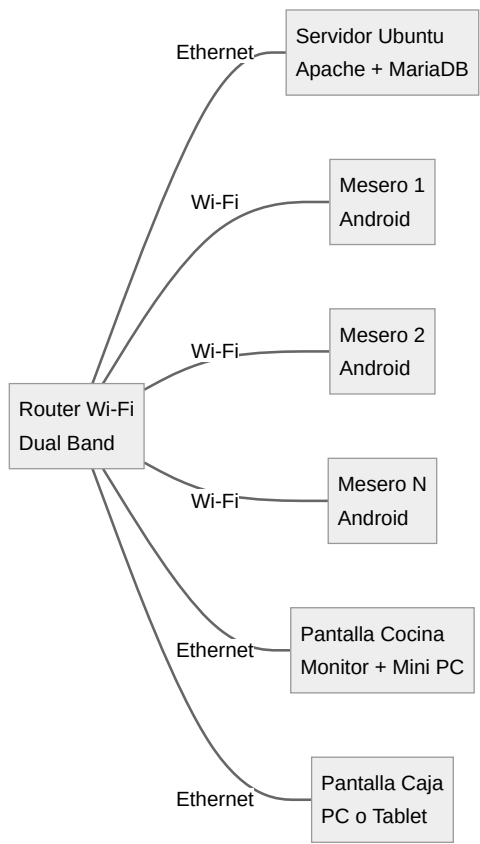


Figura 3. Topologia de red local del restaurante

7.3.2. Requisitos de Red

Tabla 13. Requisitos de Infraestructura de Red

Parametro	Especificacion
Router	Dual band (2.4 GHz y 5 GHz), minimo AC1200
Cobertura Wi-Fi	100% del area de comedor y cocina (usar repetidores si es necesario)
Rango IP	Subred privada 192.168.x.x/24, servidor con IP estatica
DNS local	Asignar nombre <code>comandas.local</code> a la IP del servidor via hosts o DNS local
Puertos	80 (HTTP), 443 (HTTPS) abiertos en firewall del servidor
Ancho de banda	Minimo 20 Mbps entre dispositivos y servidor (local)

8.1. Seguridad de Datos

- **Comunicacion cifrada:** Todas las comunicaciones entre dispositivos y servidor utilizan HTTPS con certificado SSL, incluyendo la red local.
- **Autenticacion de usuarios:** Sistema de login con contraseñas hasheadas (bcrypt) y manejo de sesiones PHP seguras.
- **Validacion de entradas:** Todos los endpoints PHP validan y sanitan los datos recibidos para prevenir inyeccion SQL y XSS.
- **Backup automatico:** Script cron que realiza dump de MariaDB cada hora hacia una ubicacion secundaria o unidad USB.

8.2. Resiliencia Operativa

- **Operacion 100% offline del reconocimiento de voz:** El motor VOSK no requiere internet en ningun momento. La transcripcion ocurre localmente en cada dispositivo.
- **Cola offline:** Si un mesero pierde la señal Wi-Fi mientras envia una comanda, esta se almacena localmente y se sincroniza automaticamente al recuperar la conexion.
- **Independencia de servicios cloud:** No hay dependencia de AWS, Google Cloud, Azure ni ningun servicio externo para el funcionamiento basico.
- **UPS recomendado:** Un interruptor de energia ininterrumpida (UPS) de 600VA o superior protege al servidor ante cortes electricos, permitiendo apagado controlado.
- **Cache del modelo:** Una vez descargado, el modelo VOSK permanece en IndexedDB incluso si el navegador se cierra o el telefono se reinicia.

8.3. Privacidad

- **Audio nunca sale del dispositivo:** Las grabaciones de voz se procesan localmente y no se transmiten ni almacenan en ningun servidor.
- **Datos almacenados minimos:** Solo se guarda el texto transcrito, la hora y los productos. No hay retencion de biometricos ni informacion sensible de clientes.
- **Control del dueño:** Todos los datos residen fisicamente en el servidor del restaurante, bajo su control exclusivo.

9. Plan de Implementacion

La implementacion del sistema se estructura en fases para minimizar el riesgo operativo y permitir una adopcion gradual por parte del personal.

Tabla 14. Cronograma de Implementacion por Fases

Fase	Duracion	Actividades	Entregable
Fase 1: Preparacion	Semana 1	Adquisicion de hardware servidor; instalacion de Ubuntu 22.04; configuracion de red local y SSL; instalacion de LAMP stack	Servidor listo para desarrollo
Fase 2: Desarrollo Backend	Semanas 2-3	Creacion de base de datos MariaDB; desarrollo de API PHP; procedimientos almacenados; autenticacion de usuarios	API funcional con CRUD completo
Fase 3: Frontend PWA	Semanas 3-4	Desarrollo de PWA del mesero; integracion de VOSK WASM; correccion fonetica; cola offline	PWA funcional en Android
Fase 4: Pantallas	Semanas 4-5	Desarrollo de pantalla de cocina (KDS); pantalla de caja; notificaciones en tiempo real	Interfaces de cocina y caja operativas
Fase 5: Catalogo	Semana 5	Carga de productos y categorias; configuracion de palabras clave para reconocimiento de voz	Catalogo completo y mapeado
Fase 6: Pruebas	Semana 6	Pruebas integrales con meseros reales; ajuste de gramatica de VOSK; optimizacion de tiempos	Sistema validado en produccion
Fase 7: Capacitacion	Semana 7	Entrenamiento de meseros en dictado por voz; capacitacion de cocineros en pantalla KDS; capacitacion de cajeros	Personal capacitado y confidente
Fase 8: Go-Live	Semana 8	Operacion en paralelo con sistema anterior (opcional); monitoreo de metricas; soporte intensivo	Sistema en produccion estable

10. Glosario de Terminos

VOSK

Toolkit de reconocimiento de voz offline de codigo abierto basado en Kaldi. Soporta mas de 20 idiomas y puede ejecutarse en dispositivos ligeros como Raspberry Pi y telefonos Android via WebAssembly.

WebAssembly (WASM)

Formato binario de instrucciones ejecutable en navegadores web a velocidades cercanas al código nativo. Permite ejecutar el motor VOSK íntegramente en el cliente sin plugins.

IndexedDB

API de almacenamiento estructurado del navegador que permite guardar grandes volúmenes de datos de forma persistente. Utilizada para cachear el modelo de lenguaje de VOSK y la cola offline de comandos.

AudioWorklet

API de procesamiento de audio de baja latencia en navegadores. Ejecuta el código de captura PCM en un hilo independiente para no bloquear la interfaz de usuario.

KDS (Kitchen Display System)

Sistema de pantalla digital para cocinas que reemplaza los tickets impresos en papel. Muestra órdenes en tiempo real con información estructurada para los cocineros.

PWA (Progressive Web App)

Aplicación web que utiliza capacidades modernas del navegador para ofrecer una experiencia similar a una app nativa: trabajo offline, push notifications, instalación en pantalla de inicio.

SSE (Server-Sent Events)

Tecnología web que permite al servidor enviar datos al navegador de forma unidireccional en tiempo real, ideal para notificar nuevas órdenes a las pantallas de cocina y caja.

Levenshtein

Algoritmo que calcula la distancia mínima de edición entre dos cadenas de texto. Utilizado para corregir errores menores de transcripción de VOSK comparando palabras contra el catálogo de productos.

Gramática restringida

Limitación del vocabulario del reconocedor de voz a un conjunto específico de palabras (productos del restaurante), lo que aumenta significativamente la precisión de transcripción.

FIFO

First In, First Out (Primero en entrar, primero en salir). Método de organización de órdenes donde la orden más antigua tiene la mayor prioridad de atención.

PHP-FPM

FastCGI Process Manager de PHP. Gestor de procesos que mejora el rendimiento de PHP al manejar multiples peticiones concurrentes de forma eficiente.

InnoDB

Motor de almacenamiento transaccional de MariaDB/MySQL que soporta ACID, claves foraneas y bloqueo a nivel de fila. Recomendado para aplicaciones con transacciones financieras.

11. Referencias

1. Alpha Cephei. (2024). *VOSK Speech Recognition Toolkit*. Documentacion oficial. <https://alphacephei.com/vosk/>
2. Alpha Cephei. (2024). *VOSK Models Repository*. Lista de modelos de lenguaje disponibles. <https://alphacephei.com/vosk/models>
3. Alphacephei. (2023). *vosk-api*. Repositorio de codigo en GitHub. <https://github.com/alphacep/vosk-api>
4. W3C. (2021). *Web Audio API*. Especificacion del World Wide Web Consortium. <https://www.w3.org/TR/webaudio/>
5. W3C. (2015). *Indexed Database API 2.0*. Especificacion del World Wide Web Consortium. <https://www.w3.org/TR/IndexedDB/>
6. Mozilla Developer Network. (2024). *Progressive Web Apps (PWAs)*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps
7. Apache Software Foundation. (2024). *Apache HTTP Server Documentation*. Version 2.4. <https://httpd.apache.org/docs/2.4/>
8. The PHP Group. (2024). *PHP Manual*. Version 8.3. <https://www.php.net/docs.php>
9. MariaDB Foundation. (2024). *MariaDB Server Documentation*. Version 10.6. <https://mariadb.com/kb/en/documentation/>
10. Ubuntu. (2024). *Ubuntu Server 22.04 LTS Documentation*. Canonical Ltd. <https://ubuntu.com/server/docs>